

第2次实验：读程序写结果

第1题：reversed(range()) 的输出顺序

以下程序的输出结果是：

```
for i in reversed(range(10, 0, -2)):
    print(i, end=" ")
```

参考答案：2 4 6 8 10 （注意10后也有空格）

解析：1. 先看内部的 range(10, 0, -2)。

range(start, stop, step) 生成序列时包含起点,不包含终点, 所以从 10 开始, 每次减 2, 到0为止

2. reversed(...)会把这个序列倒过来, 因此遍历顺序变成 2, 4, 6, 8, 10。

3. print(i, end=" ") 表示每次输出后不换行, 而是在后面加一个空格, 注意“ ”中间有一个空格。所以所有数字显示在同一行并且每一个字母后有空格, 注意10后也有空格。如果“ ”中间没有空格, 就输出246810

第2题：字符串遍历与逐项拼接

以下程序的输出结果是：

```
j = ""
for i in "12345":
    j += i + ","
print(j)
```

参考答案：1,2,3,4,5,

解析：

1. 字符串可以像序列一样逐字符遍历, for i in "12345" 中, 变量 i 依次取到 "1"、"2"、"3"、"4"、"5"。

2. j += i + "," 等价于 j = j + i + ",", 每轮都会把当前字符和一个逗号追加到原字符串末尾。

3. 最后一个字符 5 后面也会加逗号, 因为代码没有判断“是否为最后一项”。因此末尾保留一个逗号。

第3题：divmod() 同时求整商和余数

下面代码的输出结果是：

```
x = 10
y = 3
print(divmod(x, y))
```

参考答案：(3, 1)

解析：

1. `divmod(x, y)` 会一次返回两个结果，形式为 $(x // y, x \% y)$ ，第一个是整除商，第二个是余数。
2. 这里 $10 // 3 = 3$ ，因为 3 乘 3 等于 9； $10 \% 3 = 1$ ，因为 10 除以 3 余 1。
3. 所以 `divmod(10, 3)` 的结果是元组 (3, 1)。

第4题：字符串切片 a[:-2:2]

若 `a = "12345678"`，则 `print(a[:-2:2])` 的输出为：

```
a = "12345678"
print(a[:-2:2])
```

参考答案：135

解析：

1. 切片基本格式是 `s[start:stop:step]`。`a[:-2:2]` 的起始省略为空格，表示从开头开始；结束为 -2；步长为 2
2. 字符串 "12345678" 中，倒数第 2 个字符是 7。切片的结束位置不包含该位置，所以实际只在 1 到 6 之间取字符。
3. 从开头按步长 2 取，取到索引 0、2、4，对应字符 1、3、5，所以结果为 135。

第5题：find() 从指定下标开始查找

下面代码的输出结果是：

```
str1 = "Python example....wow"
str2 = "exam"
print(str1.find(str2, 5))
```

参考答案：7

解析：

1. `find()` 用来查找子串第一次出现的位置，找到返回子串起始下标，找不到返回 -1。
2. `str1.find(str2, 5)` 表示从下标 5 开始查找 "exam"。在 `Python example....wow` 中，`exam` 的首字母 `e` 位于下标 7。注意下标是从 0 开始数的。
3. 因为从下标 5 开始往下数仍然能数到 7 找到该子串，所以返回 7

